

Javid AI Project Manual

You said

WeasyPrint (HTML PDF Python)

WeasyPrint (HTML PDF Python)

Show code

Gemini said

WeasyPrint HTML/CSS PDF (Pagination) CSS Paged Media CSS

PDF !

WeasyPrint HTML/CSS PDF (Pagination) CSS Paged Media CSS

PDF !

Let's check what files are available or if we can inspect the generated PDF or HTML content from the previous run.

```
import os
```

```
print(os.listdir("src/python/pdf_exporter") if os.path.exists("src/python/pdf_exporter") else "No such dir")
```

No such dir

```
import os
```

```
print(os.getcwd())
```

```
print(os.listdir("."))
```

```
print(os.listdir("src") if os.path.exists("src") else "No src")
```

/home/bard

```
['input_file_4.png', 'input_file_3.png', 'input_file_2.png', 'input_file_1.png', 'input_file_0.png']
```

No src

Let's check what files are in /mnt/javid_data/projects/javid_ai if that's the working directory or where we were.

```

import os

target_dir = "/mnt/javid_data/projects/javid_ai"

if os.path.exists(target_dir):
    print(os.listdir(target_dir))

    sub_py = os.path.join(target_dir, "src/python/pdf_exporter")

    if os.path.exists(sub_py):
        print(os.listdir(sub_py))

# Let's check where we are or what files are around
print(os.path.exists("/mnt/javid_data/projects/javid_ai"))

False

# Let's check root or home directories
print(os.listdir("/"))

['usr', 'var', 'bin', 'lib', 'sbin', 'lib64', 'protoc-29.1-linux-x86_64.zip', 'requirements.txt', 'mnt', 'root', 'tmp', 'run',
'boot', 'dev', 'etc', 'media', 'home', 'opt', 'srv', 'sys', 'proc']

# Let's see what is in /mnt or /home/bard
print(os.listdir("/home"))
print(os.listdir("/mnt") if os.path.exists("/mnt") else "No mnt")

['bard']
['data']

```

WeasyPrint HTML/CSS PDF (Pagination) CSS Paged Media CSS

PDF !

You said

```
import os
```

```
import base64
```

```
import asyncio
```

```

from playwright.async_api import async_playwright

async def process_and_export_pdf(target_url, output_pdf_path):

    print(" [1/5] Google Chrome...")

    async with async_playwright() as p:

        try:

            browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")

            context = browser.contexts[0]

            page = context.pages[0] if context.pages else await context.new_page()

            # domcontentloaded Background Network Requests

            print(f" [2/5] : {target_url}")

            try:

                await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)

            except Exception as nav_err:

                print(f" Navigation warning (): {nav_err}")

            print(" 5 ...")

```

```
await page.wait_for_timeout(5000)
```

```
#
```

```
img_elements = await page.query_selector_all("img")
```

```
print(f" [3/5] : {len(img_elements)} ")
```

```
replaced_count = 0
```

```
for idx, img in enumerate(img_elements):
```

```
try:
```

```
box = await img.bounding_box()
```

```
if not box or box['width'] < 60 or box['height'] < 60:
```

```
continue
```

```
print(f" HD {idx + 1}...")
```

```
await img.scroll_into_view_if_needed()
```

```
await page.wait_for_timeout(500)
```

```
await img.click(force=True)
```

```
await page.wait_for_timeout(1500)
```

```
temp_img_file = f"/tmp/temp_hd_img_{idx}.png"
```

```
lightbox_img = await page.query_selector("main img, [role='main'] img, img[src*='blob:']")
```

```
if lightbox_img and await lightbox_img.is_visible():
```

```
    await lightbox_img.screenshot(path=temp_img_file)
```

```
else:
```

```
    await page.screenshot(path=temp_img_file)
```

```
back_button = await page.query_selector("button[aria-label*='Back'], button[aria-label*='']")
```

```
if back_button and await back_button.is_visible():
```

```
    await back_button.click()
```

```
else:
```

```
    await page.keyboard.press("Escape")
```

```
    await page.wait_for_timeout(800)
```

```
    with open(temp_img_file, "rb") as image_file:
```

```
        encoded_string = base64.b64encode(image_file.read()).decode('utf-8')
```

```
        base64_data_url = f"data:image/png;base64,{encoded_string}"
```

```
    await page.evaluate("""({img_elem, new_src}) => {
```

```
img_elem.src = new_src;
```

```
img_elem.srcset = "";
```

```
img_elem.style.maxWidth = '100%';
```

```
img_elem.style.height = 'auto';
```

```
img_elem.style.display = 'block';
```

```
img_elem.style.borderRadius = '8px';
```

```
}""", {"img_elem": img, "new_src": base64_data_url})
```

```
replaced_count += 1
```

```
if os.path.exists(temp_img_file):
```

```
os.remove(temp_img_file)
```

```
except Exception as e:
```

```
print(f" {idx + 1} HD : {e}")
```

```
await page.keyboard.press("Escape")
```

```
continue
```

```
print(f" [4/5] Thumbnail HD {replaced_count} ")
```

```
# =====
```

```
# [4.5/5] CSS Layout User Query &
```

```
# =====
```

```
print("[4.5/5] CSS Layout User ...")
```

```
css_content = ""
```

```
/* User */
```

```
.user-query, user-query,
```

```
div[class*="user-query"],
```

```
div[class*="request-container"] {
```

```
display: block !important;
```

```
width: 100% !important;
```

```
max-width: 100% !important;
```

```
}
```

```
/* 100% */
```

```
img {
```

max-width: 100% !important;

width: 100% !important;

height: auto !important;

display: block !important;

margin: 20px auto !important;

border-radius: 8px !important;

object-fit: contain !important;

}

/ */*

img[width="32"], img[height="32"], img[width="24"], img[height="24"],

.user-icon img, .avatar img {

width: auto !important;

display: inline-block !important;

margin: 0 !important;

}

.....

```
await page.add_style_tag(content=css_content)
```

```
await page.wait_for_timeout(1000)
```

```
print(f" [5/5] Export PDF...")
```

```
await page.pdf(
```

```
path=output_pdf_path,
```

```
format="A4",
```

```
print_background=True,
```

```
margin={"top": "15mm", "bottom": "15mm", "left": "10mm", "right": "10mm"}
```

```
)
```

```
print(f" ! PDF : {output_pdf_path}")
```

```
except Exception as err:
```

```
print(f" Chrome: {err}")
```

```
if __name__ == "__main__":
```

```
TARGET_URL = "https://gemini.google.com/app/e032b96532eefa4"
```

```
script_dir = os.path.dirname(os.path.abspath(__file__))

output_pdf = os.path.join(script_dir, "javid_smartgit_manual_hd_replaced.pdf")

asyncio.run(process_and_export_pdf(TARGET_URL, output_pdf))
```

```
import os

import base64

import asyncio

from playwright.async_api import async_playwright

async def process_and_export_pdf(target_url, output_pdf_path):

    print(" [1/5] Google Chrome...")

    async with async_playwright() as p:

        try:

            browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")

            context = browser.contexts[0]

            page = context.pages[0] if context.pages else await context.new_page()

            # domcontentloaded Background Network Requests

            print(f" [2/5] : {target_url}")

            try:

                await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
```

```

except Exception as nav_err:

print(f" Navigation warning (): {nav_err}")

print(" 5 ...")

await page.wait_for_timeout(5000)

#

img_elements = await page.query_selector_all("img")

print(f" [3/5] : {len(img_elements)} ")

replaced_count = 0

for idx, img in enumerate(img_elements):

try:

box = await img.bounding_box()

if not box or box['width'] < 60 or box['height'] < 60:

continue

print(f" HD {idx + 1}...")

await img.scroll_into_view_if_needed()

await page.wait_for_timeout(500)

await img.click(force=True)

await page.wait_for_timeout(1500)

temp_img_file = f"/tmp/temp_hd_img_{idx}.png"

lightbox_img = await page.query_selector("main img, [role='main'] img, img[src*='blob:']")

if lightbox_img and await lightbox_img.is_visible():

await lightbox_img.screenshot(path=temp_img_file)

```

else:

```
await page.screenshot(path=temp_img_file)
```

```
back_button = await page.query_selector("button[aria-label*='Back'], button[aria-label*=''])")
```

```
if back_button and await back_button.is_visible():
```

```
await back_button.click()
```

else:

```
await page.keyboard.press("Escape")
```

```
await page.wait_for_timeout(800)
```

```
with open(temp_img_file, "rb") as image_file:
```

```
encoded_string = base64.b64encode(image_file.read()).decode('utf-8')
```

```
base64_data_url = f"data:image/png;base64,{encoded_string}"
```

```
await page.evaluate("""({img_elem, new_src}) => {
```

```
img_elem.src = new_src;
```

```
img_elem.srcset = "";
```

```
img_elem.style.maxWidth = '100%';
```

```
img_elem.style.height = 'auto';
```

```
img_elem.style.display = 'block';
```

```
img_elem.style.borderRadius = '8px';
```

```
}""", {"img_elem": img, "new_src": base64_data_url})
```

```
replaced_count += 1
```

```
if os.path.exists(temp_img_file):
```

```
os.remove(temp_img_file)
```

except Exception as e:

```
print(f" {idx + 1} HD : {e}")
```

```
await page.keyboard.press("Escape")
```

```
continue
```

```
print(f" [4/5] Thumbnail HD {replaced_count} ")
```

```
# =====
```

```
# [] CSS Layout User Query &
```

```
# =====
```

```
print(" [4.5/5] CSS Layout User ...")
```

```
css_content = ""
```

```
/* User */
```

```
.user-query, user-query,
```

```
div[class*="user-query"],
```

```
div[class*="request-container"] {
```

```
display: block !important;
```

```
width: 100% !important;
```

```
max-width: 100% !important;
```

```
}
```

```
/* 100% */
```

```
img {
```

```
max-width: 100% !important;
```

```
width: 100% !important;
```

```

height: auto !important;

display: block !important;

margin: 20px auto !important;

border-radius: 8px !important;

object-fit: contain !important;

}

/* */

img[width="32"], img[height="32"], img[width="24"], img[height="24"],

.user-icon img, .avatar img {

width: auto !important;

display: inline-block !important;

margin: 0 !important;

}

.....

await page.add_style_tag(content=css_content)

await page.wait_for_timeout(1000)

print(f" [5/5] Export PDF...")

await page.pdf(

path=output_pdf_path,

format="A4",

print_background=True,

margin={"top": "15mm", "bottom": "15mm", "left": "10mm", "right": "10mm"}

```

```

)

print(f" ! PDF : {output_pdf_path}")

except Exception as err:

print(f" Chrome: {err}")

if __name__ == "__main__":

TARGET_URL = "https://gemini.google.com/app/e032b96532eefa4"

script_dir = os.path.dirname(os.path.abspath(__file__))

output_pdf = os.path.join(script_dir, "javid_smartgit_manual_hd_replaced.pdf")

asyncio.run(process_and_export_pdf(TARGET_URL, output_pdf))

```

Show code

Gemini said

! WeasyPrint PDF HTML/CSS 100% ()

Playwright HD HTML WeasyPrint PDF :

Python

```

import os

import base64

import asyncio

from playwright.async_api import async_playwright

from weasyprint import HTML, CSS

```

```

async def extract_chat_content(target_url):

    print(" [1/4] Google Chrome ...")

    async with async_playwright() as p:

        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")

```

```

context = browser.contexts[0]
page = context.pages[0] if context.pages else await context.new_page()

await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
await page.wait_for_timeout(5000)

# HTML //
# message container HTML
print(" ...")

# Element
chat_html = await page.evaluate("""() => {
    const mainContent = document.querySelector('main') || document.body;
    return mainContent.innerHTML;
}""")

return chat_html

def generate_pdf_with_weasyprint(chat_html, output_pdf_path):
    print(" [2/4] HTML Custom CSS WeasyPrint...")

    # Custom HTML Template Layout 100%
    styled_html = f"""
<!DOCTYPE html>
<html lang="th">
<head>
    <meta charset="UTF-8">
    <style>
        @page {{
            size: A4;
            margin: 20mm 15mm 20mm 15mm;
            @bottom-right {{
                content: " " counter(page) " " counter(pages);

```



```

    font-size: 10pt;
    font-family: sans-serif;
    color: #666;
  }}
}}
body {{
  font-family: 'Sarabun', 'Helvetica Neue', Arial, sans-serif;
  font-size: 11pt;
  line-height: 1.6;
  color: #222;
  background-color: #fff;
}}
/* */
img {{
  max-width: 100% !important;
  width: 100% !important;
  height: auto !important;
  display: block !important;
  margin: 15px auto !important;
  border-radius: 8px !important;
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}}
pre, code {{
  font-family: 'Courier New', Courier, monospace;
  background-color: #1e1e1e;
  color: #dcdcdc;
  padding: 10px;
  border-radius: 6px;
  display: block;
  white-space: pre-wrap;
  word-wrap: break-word;
}}
h1, h2, h3 {{

```

```

        color: #1a73e8;
        margin-top: 20px;
    }}
    .content-container {{
        width: 100% !important;
        max-width: 100% !important;
    }}
</style>
</head>
<body>
    <div class="content-container">
        {chat_html}
    </div>
</body>
</html>

```

```

print(" [3/4] PDF WeasyPrint...")
HTML(string=styled_html).write_pdf(output_pdf_path)
print(f" [4/4] PDF : {output_pdf_path}")

```

```

async def main():

```

```

    TARGET_URL = "https://gemini.google.com/app/e032b96532eefa4"
    script_dir = os.path.dirname(os.path.abspath(__file__))
    output_pdf = os.path.join(script_dir, "javid_manual_weasyprint.pdf")

```

```

    chat_html = await chat_extract_content_safe(TARGET_URL) if 'chat_extract_content_safe' in globals()
else await extract_chat_content(TARGET_URL)
    generate_pdf_with_weasyprint(chat_html, output_pdf)

```

```

if __name__ == "__main__":
    asyncio.run(main())

```

WeasyPrint Template A4 !

! WeasyPrint PDF HTML/CSS 100% ()

Playwright HD HTML WeasyPrint PDF :

Python

import os

import base64

import asyncio

from playwright.async_api import async_playwright

from weasyprint import HTML, CSS

async def extract_chat_content(target_url):

print(" [1/4] Google Chrome ...")

async with async_playwright() as p:

browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")

context = browser.contexts[0]

page = context.pages[0] if context.pages else await context.new_page()

await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)

await page.wait_for_timeout(5000)

HTML //

message container HTML

print(" ...")

Element

chat_html = await page.evaluate("""() => {

const mainContent = document.querySelector('main') || document.body;

return mainContent.innerHTML;

})""")

```
return chat_html
```

```
def generate_pdf_with_weasyprint(chat_html, output_pdf_path):
```

```
    print(" [2/4] HTML Custom CSS WeasyPrint...")
```

```
    # Custom HTML Template Layout 100%
```

```
    styled_html = f"""
```

```
    <!DOCTYPE html>
```

```
    <html lang="th">
```

```
    <head>
```

```
        <meta charset="UTF-8">
```

```
        <style>
```

```
            @page {{
```

```
                size: A4;
```

```
                margin: 20mm 15mm 20mm 15mm;
```

```
            @bottom-right {{
```

```
                content: " " counter(page) " " counter(pages);
```

```
                font-size: 10pt;
```

```
                font-family: sans-serif;
```

```
                color: #666;
```

```
            }}
```

```
        }}
```

```
        body {{
```

```
            font-family: 'Sarabun', 'Helvetica Neue', Arial, sans-serif;
```

```
            font-size: 11pt;
```

```
            line-height: 1.6;
```

```
            color: #222;
```

```
            background-color: #fff;
```

```
        }}
```

```
    /* */
```

```
    img {{
```

```
        max-width: 100% !important;
```

```

width: 100% !important;
height: auto !important;
display: block !important;
margin: 15px auto !important;
border-radius: 8px !important;
box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}}

pre, code {{
    font-family: 'Courier New', Courier, monospace;
    background-color: #1e1e1e;
    color: #dcdcdc;
    padding: 10px;
    border-radius: 6px;
    display: block;
    white-space: pre-wrap;
    word-wrap: break-word;
}}

h1, h2, h3 {{
    color: #1a73e8;
    margin-top: 20px;
}}

.content-container {{
    width: 100% !important;
    max-width: 100% !important;
}}

</style>
</head>
<body>
    <div class="content-container">
        {chat_html}
    </div>
</body>
</html>

```

.....

```
print(" [3/4] PDF WeasyPrint...")
```

```
HTML(string=styled_html).write_pdf(output_pdf_path)
```

```
print(f" [4/4] PDF : {output_pdf_path}")
```

```
async def main():
```

```
    TARGET_URL = "https://gemini.google.com/app/e032b96532eefa4"
```

```
    script_dir = os.path.dirname(os.path.abspath(__file__))
```

```
    output_pdf = os.path.join(script_dir, "javid_manual_weasyprint.pdf")
```

```
    chat_html = await chat_extract_content_safe(TARGET_URL) if 'chat_extract_content_safe' in globals()
else await extract_chat_content(TARGET_URL)
```

```
    generate_pdf_with_weasyprint(chat_html, output_pdf)
```

```
if __name__ == "__main__":
```

```
    asyncio.run(main())
```

WeasyPrint Template A4 !

Let's inspect the code provided by the user, and draft the updated WeasyPrint-based script or whatever they need.

Wait, the user wants us to adapt/modify something based on their provided script, or write the new WeasyPrint approach.

Let's see what the user pasted: their current Playwright script that replaces thumbnails with base64 HD images.

But earlier, the user agreed: "WeasyPrint (HTML PDF Python) "

And now the user says: "" (meaning: take this code / logic and adapt it using WeasyPrint or make it work according to the new plan!).

Let's write a robust Python script using WeasyPrint (or combining Playwright extraction + WeasyPrint rendering) that:

- # 1. Connects to Chrome via Playwright, extracts all text content / messages and HD images (or HTML content) cleanly.
- # 2. Generates a clean custom HTML string with proper full-width layout, nice styling, and embeds the images.
- # 3. Uses Weasyprint to render it into a polished PDF.

```
print("Ready to design the WeasyPrint script.")
```

Ready to design the WeasyPrint script.

! WeasyPrint PDF HTML/CSS 100% ()

Playwright HD HTML WeasyPrint PDF :

```
import os
```

```
import base64
```

```
import asyncio
```

```
from playwright.async_api import async_playwright
```

```
from weasyprint import HTML, CSS
```

```
async def extract_chat_content(target_url):
```

```
    print(" [1/4] Google Chrome ...")
```

```
    async with async_playwright() as p:
```

```
        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")
```

```
        context = browser.contexts[0]
```

```
        page = context.pages[0] if context.pages else await context.new_page()
```

```
        await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
```

```
        await page.wait_for_timeout(5000)
```

```
    # HTML //
```

```
    # message container HTML
```

```
    print(" ...")
```

```
    # Element
```

```

chat_html = await page.evaluate("""() => {
    const mainContent = document.querySelector('main') || document.body;
    return mainContent.innerHTML;
}""")

```

```

return chat_html

```

```

def generate_pdf_with_weasyprint(chat_html, output_pdf_path):

```

```

    print(" [2/4] HTML Custom CSS WeasyPrint...")

```

```

    # Custom HTML Template Layout 100%

```

```

    styled_html = f"""

```

```

    <!DOCTYPE html>

```

```

    <html lang="th">

```

```

    <head>

```

```

        <meta charset="UTF-8">

```

```

        <style>

```

```

            @page {{

```

```

                size: A4;

```

```

                margin: 20mm 15mm 20mm 15mm;

```

```

                @bottom-right {{

```

```

                    content: " " counter(page) " " counter(pages);

```

```

                    font-size: 10pt;

```

```

                    font-family: sans-serif;

```

```

                    color: #666;

```

```

                }}

```

```

            }}

```

```

        body {{

```

```

            font-family: 'Sarabun', 'Helvetica Neue', Arial, sans-serif;

```

```

            font-size: 11pt;

```

```

            line-height: 1.6;

```

```

            color: #222;

```

```

            background-color: #fff;

```



```

}}
/* */
img {{
    max-width: 100% !important;
    width: 100% !important;
    height: auto !important;
    display: block !important;
    margin: 15px auto !important;
    border-radius: 8px !important;
    box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}}
pre, code {{
    font-family: 'Courier New', Courier, monospace;
    background-color: #1e1e1e;
    color: #dcdcdc;
    padding: 10px;
    border-radius: 6px;
    display: block;
    white-space: pre-wrap;
    word-wrap: break-word;
}}
h1, h2, h3 {{
    color: #1a73e8;
    margin-top: 20px;
}}
.content-container {{
    width: 100% !important;
    max-width: 100% !important;
}}
</style>
</head>
<body>
<div class="content-container">

```

```

        {chat_html}
    </div>
</body>
</html>
"""

```

```

print(" [3/4] PDF WeasyPrint...")
HTML(string=styled_html).write_pdf(output_pdf_path)
print(f" [4/4] PDF : {output_pdf_path}")

```

async def main():

```

    TARGET_URL = "https://gemini.google.com/app/e032b96532eefa4"
    script_dir = os.path.dirname(os.path.abspath(__file__))
    output_pdf = os.path.join(script_dir, "javid_manual_weasyprint.pdf")

```

```

    chat_html = await chat_extract_content_safe(TARGET_URL) if 'chat_extract_content_safe' in globals()
else await extract_chat_content(TARGET_URL)
    generate_pdf_with_weasyprint(chat_html, output_pdf)

```

```

if __name__ == "__main__":
    asyncio.run(main())

```

WeasyPrint Template A4 !

You said

```

(venv) user01@JaviaAI01:/mnt/javid_data/projects/javid_ai$ python3
src/python/pdf_exporter/weasyprint_pdf.py

```

Traceback (most recent call last):

File "/mnt/javid_data/projects/javid_ai/src/python/pdf_exporter/weasyprint_pdf.py", line 5, in <module>

```

from weasyprint import HTML, CSS

```

ModuleNotFoundError: No module named 'weasyprint'

```
(venv) user01@JaviaAI01:/mnt/javid_data/projects/javid_ai$ python3  
src/python/pdf_exporter/weasyprint_pdf.py
```

Traceback (most recent call last):

File "/mnt/javid_data/projects/javid_ai/src/python/pdf_exporter/weasyprint_pdf.py", line 5, in <module>

```
from weasyprint import HTML, CSS
```

ModuleNotFoundError: No module named 'weasyprint'

Gemini said

Package weasyprint Virtual Environment !

WeasyPrint venv WeasyPrint Terminal WeasyPrint Library (cairo, pango) :

Bash

```
source venv/bin/activate
```

```
pip install weasyprint
```

:

Bash

```
python3 src/python/pdf_exporter/weasyprint_pdf.py
```

WeasyPrint !

Package weasyprint Virtual Environment !

WeasyPrint venv WeasyPrint Terminal WeasyPrint Library (cairo, pango) :

Bash

```
source venv/bin/activate
```

```
pip install weasyprint
```

:

Bash

```
python3 src/python/pdf_exporter/weasyprint_pdf.py
```

WeasyPrint !

Package weasyprint Virtual Environment !

WeasyPrint venv WeasyPrint Terminal WeasyPrint Library (cairo, pango) :

```
source venv/bin/activate
```

```
pip install weasyprint
```

:

```
python3 src/python/pdf_exporter/weasyprint_pdf.py
```

WeasyPrint !

PDF

```
javid_manual_weasyprint
```

Gemini said

! ModuleNotFoundError: No module named 'weasyprint' Virtual Environment

Terminal WeasyPrint venv :

Bash

```
source venv/bin/activate
```

```
pip install weasyprint
```

WeasyPrint :

Bash

```
python3 src/python/pdf_exporter/weasyprint_pdf.py
```

WeasyPrint PDF Template Layout !

! ModuleNotFoundError: No module named 'weasyprint' Virtual Environment

Terminal WeasyPrint venv :

Bash

```
source venv/bin/activate
```

```
pip install weasyprint
```

WeasyPrint :

Bash

```
python3 src/python/pdf_exporter/weasyprint_pdf.py
```

WeasyPrint PDF Template Layout !

! ModuleNotFoundError: No module named 'weasyprint' Virtual Environment

Terminal WeasyPrint venv :

```
source venv/bin/activate
```

```
pip install weasyprint
```

WeasyPrint :

python3 src/python/pdf_exporter/weasyprint_pdf.py

WeasyPrint PDF Template Layout !

Gemini said

! WeasyPrint PDF src/python/pdf_exporter/javid_manual_weasyprint.pdf

PDF Layout . !

! WeasyPrint PDF src/python/pdf_exporter/javid_manual_weasyprint.pdf

PDF Layout . !

! WeasyPrint PDF src/python/pdf_exporter/javid_manual_weasyprint.pdf

PDF Layout . !

Gemini said

!

mainContent.innerHTML Element UI Gemini (Component) CSS width: 100% !important !

Python (Message Container) UI PDF

extract_chat_content weasyprint_pdf.py :

Python

async def extract_chat_content(target_url):

print(" [1/4] Google Chrome ...")

async with async_playwright() as p:

browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")

context = browser.contexts[0]

```

page = context.pages[0] if context.pages else await context.new_page()

await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
await page.wait_for_timeout(5000)

# (Message container)
chat_html = await page.evaluate("""() => {
    //
    const messages = document.querySelectorAll('message-content, .message-content, user-query,
model-response');
    if (messages.length > 0) {
        let combinedHTML = "";
        messages.forEach(msg => {
            //
            const clone = msg.cloneNode(true);
            clone.querySelectorAll('img[width="32"], img[width="24"], .avatar, .user-icon').forEach(el =>
el.remove());
            combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px
solid #eee;">';
        });
        return combinedHTML;
    }
    // Fallback selector
    const mainContent = document.querySelector('main') || document.body;
    return mainContent.innerHTML;
}""")

return chat_html

```

!

!

mainContent.innerHTML Element UI Gemini (Component) CSS width: 100% !important !

Python (Message Container) UI PDF

extract_chat_content weasyprint_pdf.py :

Python

```
async def extract_chat_content(target_url):
```

```
    print(" [1/4] Google Chrome ...")
```

```
    async with async_playwright() as p:
```

```
        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")
```

```
        context = browser.contexts[0]
```

```
        page = context.pages[0] if context.pages else await context.new_page()
```

```
        await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
```

```
        await page.wait_for_timeout(5000)
```

```
    # (Message container)
```

```
    chat_html = await page.evaluate("""() => {
```

```
        //
```

```
        const messages = document.querySelectorAll('message-content, .message-content, user-query, model-response');
```

```
        if (messages.length > 0) {
```

```
            let combinedHTML = "";
```

```
            messages.forEach(msg => {
```

```
                //
```

```
                const clone = msg.cloneNode(true);
```

```
                clone.querySelectorAll('img[width="32"], img[width="24"], .avatar, .user-icon').forEach(el => el.remove());
```

```
                combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px solid #eee;">';
```

```
            });
```



```

        return combinedHTML;
    }
    // Fallback selector
    const mainContent = document.querySelector('main') || document.body;
    return mainContent.innerHTML;
}""")

```

```

return chat_html

```

!

!

mainContent.innerHTML Element UI Gemini (Component) CSS width: 100% !important !

Python (Message Container) UI PDF

extract_chat_content weasyprint_pdf.py :

```

async def extract_chat_content(target_url):

```

```

    print(" [1/4] Google Chrome ...")

```

```

    async with async_playwright() as p:

```

```

        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")

```

```

        context = browser.contexts[0]

```

```

        page = context.pages[0] if context.pages else await context.new_page()

```

```

        await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)

```

```

        await page.wait_for_timeout(5000)

```

```

    # (Message container)

```

```

    chat_html = await page.evaluate("""() => {

```

```

        //

```

```

            const messages = document.querySelectorAll('message-content, .message-content, user-query,
model-response');

```

```

if (messages.length > 0) {
  let combinedHTML = "";
  messages.forEach(msg => {
    //
    const clone = msg.cloneNode(true);
    clone.querySelectorAll('img[width="32"], img[width="24"], .avatar, .user-icon').forEach(el =>
el.remove());
    combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px
solid #eee;">';
    });
    return combinedHTML;
  }
  // Fallback selector
  const mainContent = document.querySelector('main') || document.body;
  return mainContent.innerHTML;
}""")

```

return chat_html

!

You said

Gemini said

! (Gemini Blob URL Canvas Playwright HTML)

Python Base64 String HTML WeasyPrint !

weasyprint_pdf.py Base64 :

Python

```
async def extract_chat_content(target_url):  
    print(" [1/4] Google Chrome ...")  
    async with async_playwright() as p:  
        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")  
        context = browser.contexts[0]  
        page = context.pages[0] if context.pages else await context.new_page()  
  
        await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)  
        await page.wait_for_timeout(5000)  
  
        # (Lazy loading)  
        await page.evaluate("window.scrollTo(0, document.body.scrollHeight);")  
        await page.wait_for_timeout(3000)  
        await page.evaluate("window.scrollTo(0, 0);")  
        await page.wait_for_timeout(2000)  
  
        # HTML Base64  
        chat_html = await page.evaluate("""async () => {  
            // Base64 data URL  
            const images = document.querySelectorAll('img');  
            for (let img of images) {  
                try {  
                    if (img.src && !img.src.startsWith('data:')) {  
                        const response = await fetch(img.src);  
                        const blob = await response.blob();  
                        const reader = new FileReader();  
                        const base64Promise = new Promise((resolve) => {  
                            reader.onloadend = () => resolve(reader.result);  
                            reader.readAsDataURL(blob);  
                        });  
                        img.src = await base64Promise;  
                    }  
                }  
            }  
        })
```

```

    } catch (e) {
        console.log('Image convert error:', e);
    }
}

const messages = document.querySelectorAll('message-content, .message-content, user-query,
model-response');
if (messages.length > 0) {
    let combinedHTML = "";
    messages.forEach(msg => {
        const clone = msg.cloneNode(true);
        clone.querySelectorAll('.avatar, .user-icon').forEach(el => el.remove());
        combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px
solid #ccc;">';
    });
    return combinedHTML;
}
return document.body.innerHTML;
}""")

return chat_html

```

PDF !

! (Gemini Blob URL Canvas Playwright HTML)

Python Base64 String HTML WeasyPrint !

weasyprint_pdf.py Base64 :

Python

async def extract_chat_content(target_url):

```
print(" [1/4] Google Chrome ...")
```

```
async with async_playwright() as p:
```

```
    browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")
```

```
    context = browser.contexts[0]
```

```
    page = context.pages[0] if context.pages else await context.new_page()
```

```
    await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
```

```
    await page.wait_for_timeout(5000)
```

```
    # (Lazy loading)
```

```
    await page.evaluate("window.scrollTo(0, document.body.scrollHeight);")
```

```
    await page.wait_for_timeout(3000)
```

```
    await page.evaluate("window.scrollTo(0, 0);")
```

```
    await page.wait_for_timeout(2000)
```

```
    # HTML Base64
```

```
    chat_html = await page.evaluate("""async () => {
```

```
        // Base64 data URL
```

```
        const images = document.querySelectorAll('img');
```

```
        for (let img of images) {
```

```
            try {
```

```
                if (img.src && !img.src.startsWith('data:')) {
```

```
                    const response = await fetch(img.src);
```

```
                    const blob = await response.blob();
```

```
                    const reader = new FileReader();
```

```
                    const base64Promise = new Promise((resolve) => {
```

```
                        reader.onloadend = () => resolve(reader.result);
```

```
                        reader.readAsDataURL(blob);
```

```
                    });
```

```
                    img.src = await base64Promise;
```

```
                }
```

```
            } catch (e) {
```

```
                console.log('Image convert error:', e);
```

```

    }
}

const messages = document.querySelectorAll('message-content, .message-content, user-query,
model-response');

if (messages.length > 0) {
    let combinedHTML = "";
    messages.forEach(msg => {
        const clone = msg.cloneNode(true);
        clone.querySelectorAll('.avatar, .user-icon').forEach(el => el.remove());
        combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px
solid #ccc;">';
    });
    return combinedHTML;
}

return document.body.innerHTML;
}""")

return chat_html

```

PDF !

! (Gemini Blob URL Canvas Playwright HTML)

Python Base64 String HTML WeasyPrint !

weasyprint_pdf.py Base64 :

```

async def extract_chat_content(target_url):
    print(" [1/4] Google Chrome ...")
    async with async_playwright() as p:
        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")
        context = browser.contexts[0]
        page = context.pages[0] if context.pages else await context.new_page()

```

```
await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
await page.wait_for_timeout(5000)
```

```
# (Lazy loading)
```

```
await page.evaluate("window.scrollTo(0, document.body.scrollHeight);")
await page.wait_for_timeout(3000)
await page.evaluate("window.scrollTo(0, 0);")
await page.wait_for_timeout(2000)
```

```
# HTML Base64
```

```
chat_html = await page.evaluate("""async () => {
  // Base64 data URL
  const images = document.querySelectorAll('img');
  for (let img of images) {
    try {
      if (img.src && !img.src.startsWith('data:')) {
        const response = await fetch(img.src);
        const blob = await response.blob();
        const reader = new FileReader();
        const base64Promise = new Promise((resolve) => {
          reader.onloadend = () => resolve(reader.result);
          reader.readAsDataURL(blob);
        });
        img.src = await base64Promise;
      }
    } catch (e) {
      console.log('Image convert error:', e);
    }
  }
}
```

```
const messages = document.querySelectorAll('message-content, .message-content, user-query,
model-response');
```

```

    if (messages.length > 0) {
        let combinedHTML = "";
        messages.forEach(msg => {
            const clone = msg.cloneNode(true);
            clone.querySelectorAll('.avatar, .user-icon').forEach(el => el.remove());
            combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px
solid #ccc;">';
        });
        return combinedHTML;
    }
    return document.body.innerHTML;
}""")

```

return chat_html

PDF !

PDF

javid_manual_weasyprint

Gemini said

! fetch(img.src) Blob URL Authentication Google fetch CORS / Blob

" HD " Playwright Element Base64 HTML !

extract_chat_content Base64 :

Python

```

async def extract_chat_content(target_url):
    print(" [1/4] Google Chrome HD...")
    async with async_playwright() as p:
        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")
        context = browser.contexts[0]

```



```

page = context.pages[0] if context.pages else await context.new_page()

await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
await page.wait_for_timeout(5000)

#
await page.evaluate("window.scrollTo(0, document.body.scrollHeight);")
await page.wait_for_timeout(3000)
await page.evaluate("window.scrollTo(0, 0);")
await page.wait_for_timeout(2000)

# Element
img_elements = await page.query_selector_all("img")
print( {len(img_elements)} Base64..." )

for idx, img in enumerate(img_elements):
    try:
        box = await img.bounding_box()
        #
        if not box or box['width'] < 60 or box['height'] < 60:
            continue

        await img.scroll_into_view_if_needed()
        await page.wait_for_timeout(300)

        #
        temp_img_path = f"/tmp/temp_chat_img_{idx}.png"
        await img.screenshot(path=temp_img_path)

        # Base64 string
        with open(temp_img_path, "rb") as img_file:
            encoded_string = base64.b64encode(img_file.read()).decode('utf-8')
            base64_url = f"data:image/png;base64,{encoded_string}"

```

```

# src DOM Base64

await page.evaluate("""([el, newSrc]) => {
    el.src = newSrc;
    el.removeAttribute('srcset');
    el.style.maxWidth = '100%';
    el.style.height = 'auto';
}""", [img, base64_url])

if os.path.exists(temp_img_path):
    os.remove(temp_img_path)
except Exception as img_err:
    print(f" {idx} : {img_err}")

#

chat_html = await page.evaluate("""() => {
    const messages = document.querySelectorAll('message-content, .message-content, user-query,
model-response');
    if (messages.length > 0) {
        let combinedHTML = "";
        messages.forEach(msg => {
            const clone = msg.cloneNode(true);
            clone.querySelectorAll('.avatar, .user-icon').forEach(el => el.remove());
            combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px
solid #ccc;">';
        });
        return combinedHTML;
    }
    return document.body.innerHTML;
}""")

return chat_html

```

weasyprint_pdf.py PDF !

! fetch(img.src) Blob URL Authentication Google fetch CORS / Blob

" HD " Playwright Element Base64 HTML !

extract_chat_content Base64 :

Python

```
async def extract_chat_content(target_url):
    print(" [1/4] Google Chrome HD...")
    async with async_playwright() as p:
        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")
        context = browser.contexts[0]
        page = context.pages[0] if context.pages else await context.new_page()

        await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
        await page.wait_for_timeout(5000)

        #
        await page.evaluate("window.scrollTo(0, document.body.scrollHeight);")
        await page.wait_for_timeout(3000)
        await page.evaluate("window.scrollTo(0, 0);")
        await page.wait_for_timeout(2000)

        # Element
        img_elements = await page.query_selector_all("img")
        print( {len(img_elements)} Base64...")

        for idx, img in enumerate(img_elements):
            try:
                box = await img.bounding_box()
```

```
#
```

```
if not box or box['width'] < 60 or box['height'] < 60:
```

```
    continue
```

```
await img.scroll_into_view_if_needed()
```

```
await page.wait_for_timeout(300)
```

```
#
```

```
temp_img_path = f"/tmp/temp_chat_img_{idx}.png"
```

```
await img.screenshot(path=temp_img_path)
```

```
# Base64 string
```

```
with open(temp_img_path, "rb") as img_file:
```

```
    encoded_string = base64.b64encode(img_file.read()).decode('utf-8')
```

```
    base64_url = f"data:image/png;base64,{encoded_string}"
```

```
# src DOM Base64
```

```
await page.evaluate("""([el, newSrc]) => {
```

```
    el.src = newSrc;
```

```
    el.removeAttribute('srcset');
```

```
    el.style.maxWidth = '100%';
```

```
    el.style.height = 'auto';
```

```
}""", [img, base64_url])
```

```
if os.path.exists(temp_img_path):
```

```
    os.remove(temp_img_path)
```

```
except Exception as img_err:
```

```
    print(f" {idx} : {img_err}")
```

```
#
```

```
chat_html = await page.evaluate("""() => {
```

```
    const messages = document.querySelectorAll('message-content, .message-content, user-query,
model-response');
```

```

    if (messages.length > 0) {
        let combinedHTML = "";
        messages.forEach(msg => {
            const clone = msg.cloneNode(true);
            clone.querySelectorAll('.avatar, .user-icon').forEach(el => el.remove());
            combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px
solid #ccc;">';
        });
        return combinedHTML;
    }
    return document.body.innerHTML;
}""")

return chat_html

```

weasyprint_pdf.py PDF !

! fetch(img.src) Blob URL Authentication Google fetch CORS / Blob

" HD " Playwright Element Base64 HTML !

extract_chat_content Base64 :

```

async def extract_chat_content(target_url):
    print(" [1/4] Google Chrome HD...")
    async with async_playwright() as p:
        browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")
        context = browser.contexts[0]
        page = context.pages[0] if context.pages else await context.new_page()

        await page.goto(target_url, wait_until="domcontentloaded", timeout=30000)
        await page.wait_for_timeout(5000)

#

```

```

await page.evaluate("window.scrollTo(0, document.body.scrollHeight);")
await page.wait_for_timeout(3000)
await page.evaluate("window.scrollTo(0, 0);")
await page.wait_for_timeout(2000)

```

```
# Element
```

```

img_elements = await page.query_selector_all("img")
print( {len(img_elements)} Base64...")

```

```
for idx, img in enumerate(img_elements):
```

```
    try:
```

```
        box = await img.bounding_box()
```

```
        #
```

```
        if not box or box['width'] < 60 or box['height'] < 60:
```

```
            continue
```

```
        await img.scroll_into_view_if_needed()
```

```
        await page.wait_for_timeout(300)
```

```
        #
```

```
        temp_img_path = f"/tmp/temp_chat_img_{idx}.png"
```

```
        await img.screenshot(path=temp_img_path)
```

```
    # Base64 string
```

```
    with open(temp_img_path, "rb") as img_file:
```

```
        encoded_string = base64.b64encode(img_file.read()).decode('utf-8')
```

```
        base64_url = f"data:image/png;base64,{encoded_string}"
```

```
    # src DOM Base64
```

```
    await page.evaluate("""([el, newSrc]) => {
```

```
        el.src = newSrc;
```

```
        el.removeAttribute('srcset');
```

```
        el.style.maxWidth = '100%';
```

```
el.style.height = 'auto';
}""", [img, base64_url])
```

```
if os.path.exists(temp_img_path):
    os.remove(temp_img_path)
except Exception as img_err:
    print(f" {idx} : {img_err}")
```

```
#
```

```
chat_html = await page.evaluate("""() => {
    const messages = document.querySelectorAll('message-content, .message-content, user-query,
model-response');
    if (messages.length > 0) {
        let combinedHTML = "";
        messages.forEach(msg => {
            const clone = msg.cloneNode(true);
            clone.querySelectorAll('.avatar, .user-icon').forEach(el => el.remove());
            combinedHTML += clone.outerHTML + '<hr style="margin: 20px 0; border: 0; border-top: 1px
solid #ccc;">';
        });
        return combinedHTML;
    }
    return document.body.innerHTML;
}""")
```

```
return chat_html
```

weasyprint_pdf.py PDF !

You said

```
(venv) user01@JaviaAI01:/mnt/javid_data/projects/javid_ai$
src/python/pdf_exporter/weasyprint_pdf.py
```

python3

File "/mnt/javid_data/projects/javid_ai/src/python/pdf_exporter/weasyprint_pdf.py", line 25

```
print( {len(img_elements)} Base64...")
```

^

SyntaxError: unterminated string literal (detected at line 25)

```
(venv) user01@JaviaAI01:/mnt/javid_data/projects/javid_ai$ python3
src/python/pdf_exporter/weasyprint_pdf.py
```

File "/mnt/javid_data/projects/javid_ai/src/python/pdf_exporter/weasyprint_pdf.py", line 25

```
print( {len(img_elements)} Base64...")
```

^

SyntaxError: unterminated string literal (detected at line 25)

Gemini said

f-string print f Python string

:

Python

```
print(f" {len(img_elements)} Base64...")
```

!

f-string print f Python string

:

Python


```
print(f" {len(img_elements)} Base64...")
```

!

f-string print f Python string

:

```
print(f" {len(img_elements)} Base64...")
```

!

You said

Gemini said

! 22

```
img.screenshot() 100% UI (Delay)
```

extract_chat_content :

Python

```
for idx, img in enumerate(img_elements):  
    try:  
        box = await img.bounding_box()  
        if not box or box['width'] < 60 or box['height'] < 60:  
            continue  
  
        await img.scroll_into_view_if_needed()  
        await page.wait_for_timeout(800) #
```

```
temp_img_path = f'/tmp/temp_chat_img_{idx}.png'
```

```
await img.screenshot(path=temp_img_path)
```

```
with open(temp_img_path, "rb") as img_file:
```

```
    encoded_string = base64.b64encode(img_file.read()).decode('utf-8')
```

```
    base64_url = f"data:image/png;base64,{encoded_string}"
```

```
await page.evaluate("""([el, newSrc]) => {
```

```
    el.src = newSrc;
```

```
    el.removeAttribute('srcset');
```

```
    el.style.maxWidth = '100%';
```

```
    el.style.height = 'auto';
```

```
    el.style.display = 'block';
```

```
    el.style.margin = '10px auto';
```

```
}""", [img, base64_url])
```

```
if os.path.exists(temp_img_path):
```

```
    os.remove(temp_img_path)
```

```
except Exception as img_err:
```

```
    print(f" {idx} : {img_err}")
```

!

! 22

img.screenshot() 100% UI (Delay)

extract_chat_content :

Python

```
for idx, img in enumerate(img_elements):
```

try:

```
box = await img.bounding_box()
if not box or box['width'] < 60 or box['height'] < 60:
    continue
```

```
await img.scroll_into_view_if_needed()
await page.wait_for_timeout(800) #
```

```
temp_img_path = f"/tmp/temp_chat_img_{idx}.png"
await img.screenshot(path=temp_img_path)
```

```
with open(temp_img_path, "rb") as img_file:
    encoded_string = base64.b64encode(img_file.read()).decode('utf-8')
    base64_url = f"data:image/png;base64,{encoded_string}"
```

```
await page.evaluate("""([el, newSrc]) => {
    el.src = newSrc;
    el.removeAttribute('srcset');
    el.style.maxWidth = '100%';
    el.style.height = 'auto';
    el.style.display = 'block';
    el.style.margin = '10px auto';
}""", [img, base64_url])
```

```
if os.path.exists(temp_img_path):
    os.remove(temp_img_path)
```

```
except Exception as img_err:
    print(f" {idx} : {img_err}")
```

!

! 22

```
img.screenshot() 100% UI (Delay)
```

```
extract_chat_content :
```

```
for idx, img in enumerate(img_elements):
```

```
    try:
```

```
        box = await img.bounding_box()
```

```
        if not box or box['width'] < 60 or box['height'] < 60:
```

```
            continue
```

```
        await img.scroll_into_view_if_needed()
```

```
        await page.wait_for_timeout(800) #
```

```
        temp_img_path = f"/tmp/temp_chat_img_{idx}.png"
```

```
        await img.screenshot(path=temp_img_path)
```

```
        with open(temp_img_path, "rb") as img_file:
```

```
            encoded_string = base64.b64encode(img_file.read()).decode('utf-8')
```

```
            base64_url = f"data:image/png;base64,{encoded_string}"
```

```
        await page.evaluate("""([el, newSrc]) => {
```

```
            el.src = newSrc;
```

```
            el.removeAttribute('srcset');
```

```
            el.style.maxWidth = '100%';
```

```
            el.style.height = 'auto';
```

```
            el.style.display = 'block';
```

```
            el.style.margin = '10px auto';
```

```
        }""", [img, base64_url])
```

```
        if os.path.exists(temp_img_path):
```

```
            os.remove(temp_img_path)
```

```
    except Exception as img_err:
```

```
        print(f" {idx} : {img_err}")
```

!

Gemini is AI and can make mistakes.